

Week1_Gueorgui_Poklitar

June 28, 2026

```
[1]: #pip install pandas numpy scikit-learn matplotlib seaborn statsmodels kagglehub
```

1 Week 1 - Linear Regression 1

```
[2]: #pip install pandas numpy scikit-learn matplotlib seaborn statsmodels kagglehub
```

```
[3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import kagglehub

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler,
    ↳OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import r2_score, root_mean_squared_error

from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

print("All imports successful.")
```

All imports successful.

1.1 1. Data Loading & Cleaning

```
[4]: #Load from Kaggle
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")
```

```

#Cleaning
df = df.drop_duplicates()

# Median imputation for employment length (right-skewed, so median > mean here)
df['person_emp_length'] = df['person_emp_length'].
    ↪fillna(df['person_emp_length'].median())

# Grade-matched median for interest rate - preserves credit-grade logic
df['loan_int_rate'] = df['loan_int_rate'].fillna(
    df.groupby('loan_grade')['loan_int_rate'].transform('median')
)

# Remove logical impossibilities (not just statistical outliers)
df = df[df['person_emp_length'] <= df['person_age']]
df = df[df['person_age'] <= 100]
df = df[df['person_emp_length'] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate (loan_status=1): {df['loan_status'].mean()*100:.2f}%")
print(f"\nColumn types:\n{df.dtypes}")

```

Downloading to /home/codespace/.cache/kagglehub/datasets/laotse/credit-risk-dataset/1.archive...

100%| | 368k/368k [00:00<00:00, 28.7MB/s]

Extracting files...

Raw shape: (32581, 12)

Clean shape: (32409, 12)

Default rate (loan_status=1): 21.87%

Column types:

person_age	int64
person_income	int64
person_home_ownership	str
person_emp_length	float64
loan_intent	str
loan_grade	str
loan_amnt	int64
loan_int_rate	float64
loan_status	int64
loan_percent_income	float64
cb_person_default_on_file	str
cb_person_cred_hist_length	int64
dtype:	object

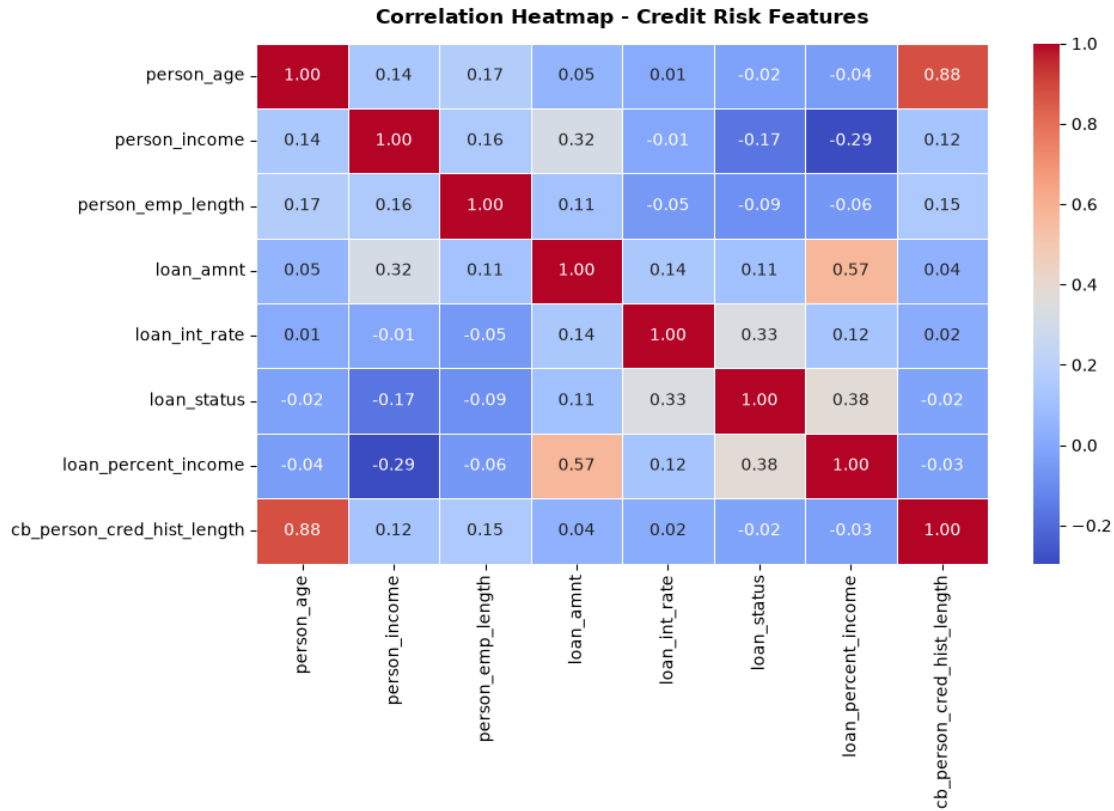
1.2 2. Exploratory Baseline: Correlation & Feature Context

```
[5]: numeric_cols = ['person_age', 'person_income', 'person_emp_length',
                    'loan_amnt', 'loan_int_rate', 'loan_status',
                    'loan_percent_income', 'cb_person_cred_hist_length']

corr = df[numeric_cols].corr()

fig, ax = plt.subplots(figsize=(10, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            linewidths=0.5, ax=ax)
ax.set_title('Correlation Heatmap - Credit Risk Features', fontweight='bold',
            pad=12)
plt.tight_layout()
plt.show()

print("Key correlations with loan_int_rate:")
print(corr['loan_int_rate'].sort_values(ascending=False).to_string())
print()
print("Interpretation: loan_int_rate correlates most strongly with
    loan_percent_income")
print("and loan_amnt among the numeric features. The strongest single driver -
    loan_grade -")
print("is categorical and does not appear here; it enters the model in Section
    3.")
```



Key correlations with loan_int_rate:

```

loan_int_rate      1.000000
loan_status        0.334110
loan_amnt          0.142399
loan_percent_income 0.120515
cb_person_cred_hist_length 0.015048
person_age         0.010860
person_income      -0.005607
person_emp_length  -0.053000

```

Interpretation: loan_int_rate correlates most strongly with loan_percent_income and loan_amnt among the numeric features. The strongest single driver - loan_grade - is categorical and does not appear here; it enters the model in Section 3.

1.3 3. Categorical & Continuous Features: Baseline OLS with Full Pipeline

```

[6]: # Define features
categorical_features = ['loan_intent', 'loan_grade',
                        'person_home_ownership', 'cb_person_default_on_file']
numeric_features = ['person_income', 'person_age', 'person_emp_length',

```

```

        'loan_amnt', 'loan_percent_income']

X = df[categorical_features + numeric_features].copy()
y = df['loan_int_rate'].copy()

mask = X.notna().all(axis=1)
X, y = X[mask], y[mask]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Build preprocessing pipeline: scale numerics, one-hot encode categoricals
preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
    categorical_features)
])

ols_pipeline = Pipeline(steps=[
    ('prep', preprocessor),
    ('model', LinearRegression())
])

ols_pipeline.fit(X_train, y_train)
preds_ols = ols_pipeline.predict(X_test)

r2_ols = r2_score(y_test, preds_ols)
rmse_ols = root_mean_squared_error(y_test, preds_ols)

print("Baseline OLS (categorical + continuous features)")
print(f"  Test R-squared : {r2_ols:.4f}")
print(f"  Test RMSE      : {rmse_ols:.4f} percentage points")
print()
print("Interpretation: A high R-squared here is driven almost entirely by
    loan_grade,")
print("which the lender assigns in lockstep with the rate. The continuous
    features")
print("(income, age, loan size) contribute the remaining, smaller slice of
    explained")
print("variance. Week 3 revisits this exact target WITHOUT loan_grade to force
    an honest model.")

```

```

Baseline OLS (categorical + continuous features)
  Test R-squared : 0.9099
  Test RMSE      : 0.9678 percentage points

```

Interpretation: A high R-squared here is driven almost entirely by loan_grade, which the lender assigns in lockstep with the rate. The continuous features (income, age, loan size) contribute the remaining, smaller slice of explained variance. Week 3 revisits this exact target WITHOUT loan_grade to force an honest model.

1.4 4. Multicollinearity & Variance Inflation Factor (VIF)

```
[ ]: #VIF is computed on numeric-only features
vif_features = ['person_age', 'person_income', 'person_emp_length',
                'loan_amnt', 'loan_percent_income',
                'cb_person_cred_hist_length', 'loan_int_rate']

df_vif = df[vif_features].dropna()
X_vif = add_constant(df_vif)

vif_data = pd.DataFrame({
    'Feature': X_vif.columns,
    'VIF':      [variance_inflation_factor(X_vif.values, i)
                 for i in range(X_vif.shape[1])]
}).query("Feature != 'const'").sort_values('VIF', ascending=False)

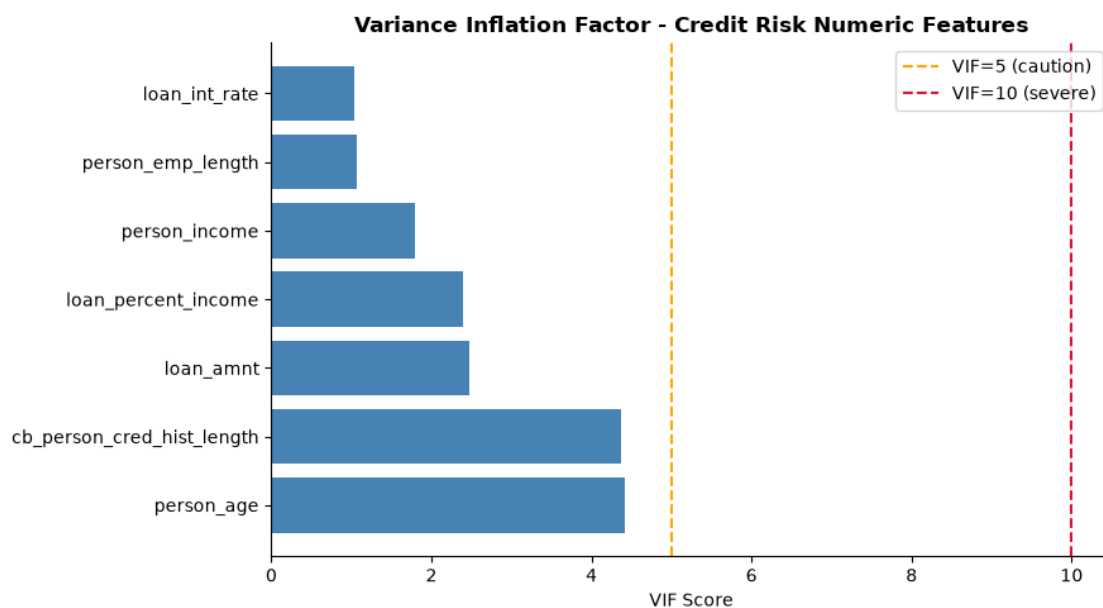
print(vif_data.to_string(index=False))
print()
print("Finding: person_age and cb_person_cred_hist_length show elevated VIF_
    ↳because")
print("they are nearly redundant (credit history length grows mechanically with_
    ↳age).")
print("In all downstream models, cb_person_cred_hist_length is dropped to_
    ↳avoid")
print("coefficient instability - this is the rule established here and reused_
    ↳in Weeks 2-3.")

#VIF Bar Chart
fig, ax = plt.subplots(figsize=(9, 5))
colors_vif = ['crimson' if v > 10 else 'orange' if v > 5 else 'steelblue'
              for v in vif_data['VIF']]
ax.barh(vif_data['Feature'], vif_data['VIF'], color=colors_vif)
ax.axvline(5, color='orange', linestyle='--', linewidth=1.5, label='VIF=5_
    ↳(caution)')
ax.axvline(10, color='crimson', linestyle='--', linewidth=1.5, label='VIF=10_
    ↳(severe)')
ax.set_xlabel('VIF Score')
ax.set_title('Variance Inflation Factor - Credit Risk Numeric Features',_
    ↳fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
```

```
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()
```

Feature	VIF
person_age	4.421454
cb_person_cred_hist_length	4.375880
loan_amnt	2.471783
loan_percent_income	2.402678
person_income	1.799407
person_emp_length	1.067159
loan_int_rate	1.028640

Finding: person_age and cb_person_cred_hist_length show elevated VIF because they are nearly redundant (credit history length grows mechanically with age). In all downstream models, cb_person_cred_hist_length is dropped to avoid coefficient instability - this is the rule established here and reused in Weeks 2-3.



1.5 5. Polynomial Terms

```
[ ]: #Using the two most rate-relevant numeric features for a clean polynomial test
poly_features = ['loan_percent_income', 'loan_amnt']

df_poly = df[poly_features + ['loan_int_rate']].dropna()
X_poly_raw = df_poly[poly_features]
y_poly      = df_poly['loan_int_rate']
```

```

X_poly_train, X_poly_test, yp_train, yp_test = train_test_split(
    X_poly_raw, y_poly, test_size=0.2, random_state=42
)

results_poly = []
for deg in [1, 2, 3]:
    pipe = Pipeline([
        ('poly', PolynomialFeatures(degree=deg, include_bias=False)),
        ('scaler', StandardScaler()),
        ('model', LinearRegression())
    ])
    pipe.fit(X_poly_train, yp_train)
    preds_p = pipe.predict(X_poly_test)
    results_poly.append({
        'Degree': deg,
        'R-squared': round(r2_score(yp_test, preds_p), 4),
        'RMSE': round(root_mean_squared_error(yp_test, preds_p), 4),
        'N Features': PolynomialFeatures(degree=deg).fit(X_poly_raw).
        ↪n_output_features_
    })

poly_df = pd.DataFrame(results_poly)
print("Polynomial Regression - loan_percent_income & loan_amnt to_
    ↪loan_int_rate")
print(poly_df.to_string(index=False))

#Visual: scatter + polynomial fit (1D: loan_percent_income only)
df_1d = df[['loan_percent_income', 'loan_int_rate']].dropna()
df_1d = df_1d[df_1d['loan_percent_income'] < 0.8]
sample_1d = df_1d.sample(min(3000, len(df_1d)), random_state=42)

x_range = np.linspace(sample_1d['loan_percent_income'].min(),
                        sample_1d['loan_percent_income'].max(), 300).
    ↪reshape(-1,1)

fig, ax = plt.subplots(figsize=(9, 5))
ax.scatter(sample_1d['loan_percent_income'], sample_1d['loan_int_rate'],
           alpha=0.15, s=8, color='steelblue', label='Observations')

colors_deg = ['gray', 'darkorange', 'crimson']
for deg, col in zip([1, 2, 3], colors_deg):
    pipe_1d = Pipeline([
        ('poly', PolynomialFeatures(degree=deg, include_bias=False)),
        ('model', LinearRegression())
    ])
    pipe_1d.fit(sample_1d[['loan_percent_income']], sample_1d[['loan_int_rate']])

```



```

ax.plot(x_range, pipe_1d.predict(x_range), color=col, linewidth=2.5,
        label=f'Degree {deg}')

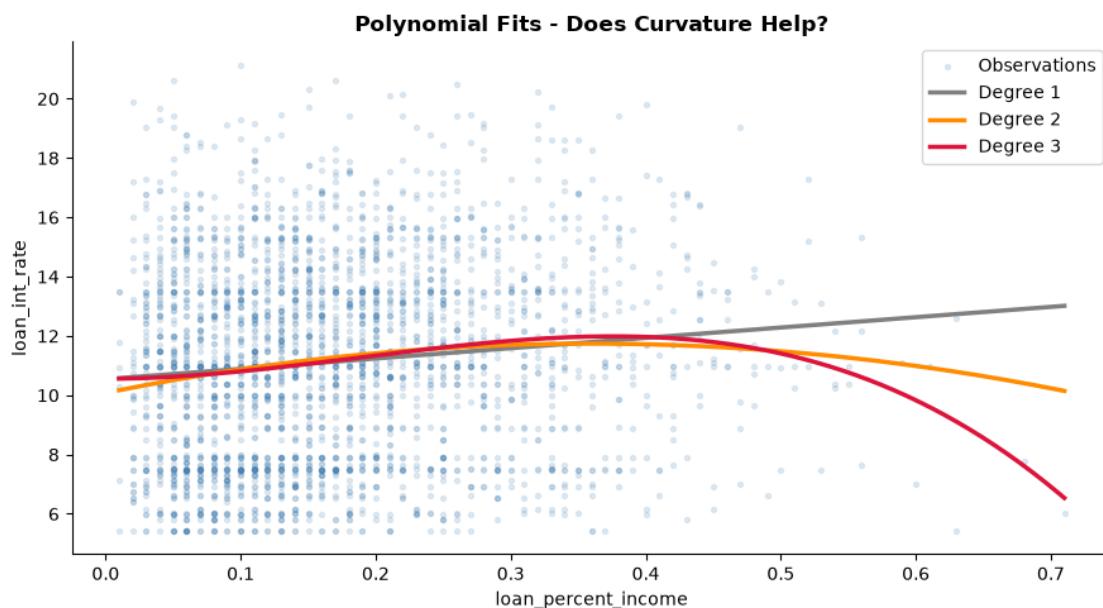
ax.set_xlabel('loan_percent_income')
ax.set_ylabel('loan_int_rate')
ax.set_title('Polynomial Fits - Does Curvature Help?', fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: The marginal R-squared gain from degree 2 or 3 is tiny.␣
↪The")
print("rate-vs-debt-burden relationship is essentially linear; added curvature␣
↪mostly")
print("fits noise. This justifies keeping linear terms in the main model and␣
↪treating")
print("polynomial expansion as a tested-and-rejected hypothesis, not a default.
↪")

```

Polynomial Regression - loan_percent_income & loan_amnt to loan_int_rate

Degree	R-squared	RMSE	N Features
1	0.0220	3.1888	3
2	0.0270	3.1806	6
3	0.0372	3.1640	10



Interpretation: The marginal R-squared gain from degree 2 or 3 is tiny. The rate-vs-debt-burden relationship is essentially linear; added curvature mostly fits noise. This justifies keeping linear terms in the main model and treating polynomial expansion as a tested-and-rejected hypothesis, not a default.

1.6 6. Interaction Terms

```
[ ]: df_int = df[['person_income', 'person_emp_length', 'loan_int_rate']].dropna().
      ↪copy()

#Manually create interaction term for transparency
df_int['income_x_emp'] = df_int['person_income'] * df_int['person_emp_length']

Xi_train, Xi_test, yi_train, yi_test = train_test_split(
    df_int[['person_income', 'person_emp_length', 'income_x_emp']],
    df_int['loan_int_rate'], test_size=0.2, random_state=42
)

results_int = []
for label, feats in [('No Interaction', ['person_income', 'person_emp_length']),
                    ('With Interaction',
                     ↪['person_income', 'person_emp_length', 'income_x_emp'])]:
    sc_i = StandardScaler()
    X_tr_i = sc_i.fit_transform(Xi_train[feats])
    X_te_i = sc_i.transform(Xi_test[feats])
    lr_i = LinearRegression().fit(X_tr_i, yi_train)
    preds_i = lr_i.predict(X_te_i)
    results_int.append({
        'Model': label,
        'R-squared': round(r2_score(yi_test, preds_i), 4),
        'RMSE': round(root_mean_squared_error(yi_test, preds_i), 4)
    })
    if label == 'With Interaction':
        print("Coefficients (standardized):")
        for name, c in zip(feats, lr_i.coef_):
            print(f" {name:<30}: {c:.5f}")
        print(f" {'Intercept':<30}: {lr_i.intercept_:.4f}")

print()
print(pd.DataFrame(results_int).to_string(index=False))
print()
print("Interpretation: If income_x_emp carried a meaningful coefficient, the
      ↪marginal")
print("effect of income on rate would depend on employment stability. A small
      ↪interaction")
```

```
print("coefficient and a flat R-squared confirm that income and employment act_
↳mostly")
print("independently for rate-setting - so the simpler additive model is_
↳preferred.")
```

Coefficients (standardized):

```
person_income      : -0.11499
person_emp_length  : -0.27238
income_x_emp       : 0.19563
Intercept          : 11.0270
```

	Model	R-squared	RMSE
No Interaction		0.0036	3.2187
With Interaction		0.0034	3.2190

Interpretation: If income_x_emp carried a meaningful coefficient, the marginal effect of income on rate would depend on employment stability. A small interaction coefficient and a flat R-squared confirm that income and employment act mostly independently for rate-setting - so the simpler additive model is preferred.

1.7 7. Bringing It Together: OLS with Polynomial + Interaction + Categorical

```
[10]: # A single pipeline that combines everything from Sections 3-6:
#      - scaled continuous features WITH degree-2 polynomial + interaction_
↳expansion
#      - one-hot encoded categoricals
#      - cb_person_cred_hist_length deliberately excluded (VIF rule from Section 4)
poly_num = ['person_income', 'person_age', 'person_emp_length',
            'loan_amnt', 'loan_percent_income']
cat_feats = ['loan_intent', 'loan_grade', 'person_home_ownership',
↳'cb_person_default_on_file']

X = df[poly_num + cat_feats].copy()
y = df['loan_int_rate'].copy()
mask = X.notna().all(axis=1)
X, y = X[mask], y[mask]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)

num_poly_pipe = Pipeline([
    ('poly', PolynomialFeatures(degree=2, include_bias=False,
↳interaction_only=False)),
    ('scale', StandardScaler())
])
full_prep = ColumnTransformer([
```

```

        ('numpoly', num_poly_pipe, poly_num),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
        ↪cat_feats)
    ])

models = {
    'Simple OLS (linear, no poly)': Pipeline([
        ('prep', ColumnTransformer([
            ('num', StandardScaler(), poly_num),
            ('cat', OneHotEncoder(handle_unknown='ignore',
        ↪sparse_output=False), cat_feats)])),
        ('lr', LinearRegression())]),
    'Full OLS (+ poly2 + interactions)': Pipeline([
        ('prep', full_prep), ('lr', LinearRegression())]),
}]

rows = []
for name, m in models.items():
    m.fit(X_train, y_train)
    p = m.predict(X_test)
    cv = cross_val_score(m, X_train, y_train, cv=5, scoring='r2')
    rows.append({'Model': name,
                'Test R-squared': round(r2_score(y_test, p), 4),
                'Test RMSE': round(root_mean_squared_error(y_test, p), 4),
                'CV R-squared Mean': round(cv.mean(), 4),
                'CV R-squared Std': round(cv.std(), 4)})

compare = pd.DataFrame(rows)
print(compare.to_string(index=False))

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.bar(['Simple OLS', 'Full OLS\n(+poly2 +interactions)'], compare['Test
    ↪R-squared'],
        color=['#555555', 'steelblue'], edgecolor='white')
for i, v in enumerate(compare['Test R-squared']):
    ax.text(i, v + 0.002, f'{v:.4f}', ha='center', fontsize=10,
    ↪fontweight='bold')
ax.set_ylabel('Test R-squared')
ax.set_title('Does Polynomial + Interaction Expansion Earn Its Complexity?',
    ↪fontweight='bold')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()

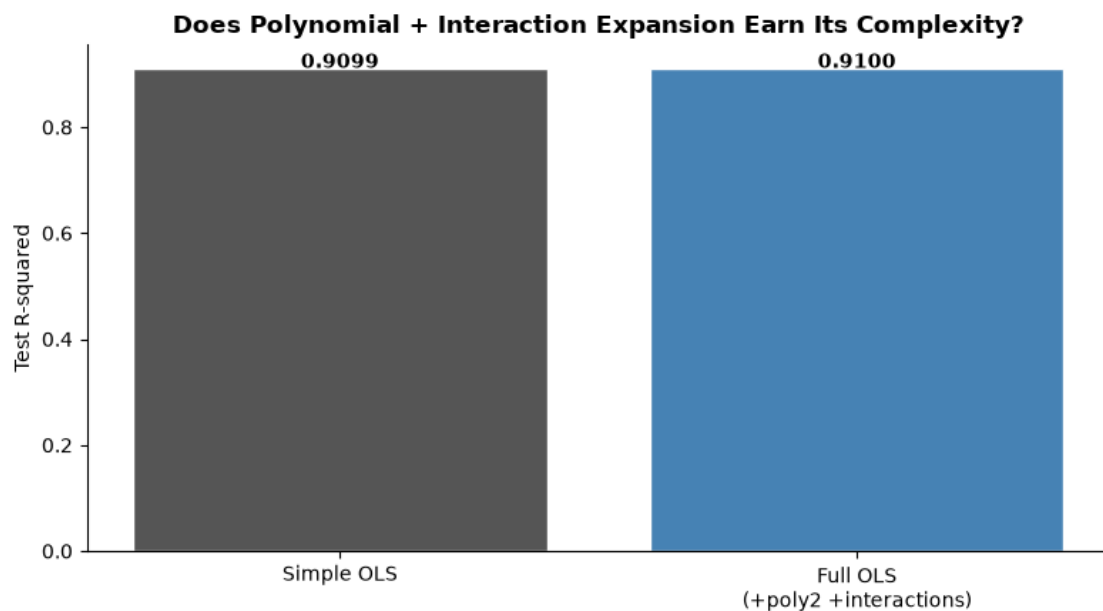
```

```

print("What this means for the Chief Risk Officer: the elaborate feature_
    ↪expansion buys")
print("almost nothing over the simple linear model. The honest recommendation_
    ↪is to keep")
print("the parsimonious specification - it is easier to audit, explain to_
    ↪regulators, and")
print("less prone to overfitting. Complexity must justify itself; here it does_
    ↪not.")

```

	Model	Test R-squared	Test RMSE	CV R-squared	Mean
CV R-squared Std					
	Simple OLS (linear, no poly)	0.9099	0.9678		0.9115
0.001					
	Full OLS (+ poly2 + interactions)	0.9100	0.9671		0.9116
0.001					



What this means for the Chief Risk Officer: the elaborate feature expansion buys almost nothing over the simple linear model. The honest recommendation is to keep the parsimonious specification - it is easier to audit, explain to regulators, and less prone to overfitting. Complexity must justify itself; here it does not.

1.7.1 Week 1 Takeaways and the Bridge to Week 2

What this week established: - A reusable cleaning recipe (median imputation, grade-matched rate imputation, removal of logical impossibilities) that every later notebook inherits unchanged.

- The OLS baseline for predicting `loan_int_rate`, and the finding that `loan_grade` dominates explained variance because the lender assigns rate *from* grade. - A multicollinearity rule: `cb_person_cred_hist_length` is dropped because it is nearly collinear with `person_age` (high VIF). - Tested-and-rejected complexity: neither polynomial terms nor an income x employment interaction meaningfully improve fit. The data want a simple, additive linear model.